Práctica 05: Secuencias y algoritmos básicos

Elaborado por:	Revisado por:	Autorizado por:
Ing. Mayelly Reynoso Andrade	Ing. Jorge Luis López García	Dra. Rocío Alejandra Aldeco Pérez

Práctica 05: Secuencias y algoritmos básicos

Objetivo:

Aplicar el pensamiento algorítmico para construir soluciones mediante secuencias ordenadas de instrucciones, con el propósito de representar procesos de forma clara, lógica y ejecutable.

Introducción:

En la resolución de problemas, es fundamental contar con procedimientos claros que permitan transformar datos de entrada en resultados útiles. El pensamiento algorítmico responde a esta necesidad mediante la construcción de secuencias ordenadas de instrucciones, donde cada paso se ejecuta de manera lógica y sin ambigüedades.[1]

Las secuencias constituyen la forma más básica de un algoritmo, ya que describen procesos lineales en los que las acciones se realizan una tras otra. Este tipo de estructura es esencial para modelar problemas mediante representaciones como pseudocódigo y diagramas de flujo, en los que se establecen pasos ordenados sin ramificaciones ni interrupciones del flujo de ejecución.[2]

En este argumento, un problema se entiende como una situación que requiere la transformación de un conjunto de datos de entrada en resultados específicos mediante un procedimiento bien definido. El análisis del problema consiste en identificar sus elementos esenciales, tales como entradas, procesos y salidas, con el fin de estructurar una solución lógica antes de su implementación. El proceso en secuencia se refiere a la descripción ordenada de operaciones utilizando operadores aritméticos, relacionales y lógicos para transformar los datos paso a paso, sin el uso de funciones intrínsecas, garantizando así una representación explícita de cada operación dentro del algoritmo.

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
[2] Joyanes Aguilar, L. (2020). Fundamentos de Programación: Algoritmos, estructuras de datos y objetos (5.ª ed.). McGraw-Hill.

Asimismo, la prueba de escritorio es una técnica manual que permite simular la ejecución de un algoritmo paso a paso, con distintos valores de entrada, con el propósito de verificar su corrección lógica y detectar posibles errores antes de su implementación en un lenguaje de programación.[1]

La construcción de algoritmos debe partir de un análisis claro del problema y avanzar hacia una solución estructurada mediante pasos lógicos bien definidos. Asimismo, se establece que antes de programar es indispensable definir con precisión la secuencia de operaciones que transforman los datos de entrada en resultados. En esta práctica, se desarrollará la capacidad de diseñar algoritmos secuenciales, verificar su funcionamiento mediante pruebas de escritorio y reconocer su importancia en la resolución de problemas computacionales.

Tabla de operadores matemáticos

Operador	Significado	Descripción
+	Suma	Realiza la adición de dos valores numéricos
-	Resta	Calcula la diferencia entre dos valores
*****	Multiplicación	Multiplica dos operandos
/	División	Divide un valor entre otro
MOD	Módulo (residuo)	Devuelve el residuo de una división entera
DIV	División entera	Retorna la parte entera del cociente
^ o **	Potencia	Eleva un valor (base) a la potencia de otro (exponente)

Tabla 1. Operadores aritméticos.

Tabla de operadores relacionales

Operador	Significado	Descripción
=	Igualdad	Compara si dos valores son iguales
<> o !=	Diferente	Evalúa si dos valores son distintos
>	Mayor que	Compara si un valor es mayor que otro
<	Menor que	Compara si un valor es menor que otro
>=	Mayor o igual	Evalúa si un valor es mayor o igual
<=	Menor o igual	Evalúa si un valor es menor o igual

Tabla 2. Operadores relacionales.

[1] Knuth, D. E. (1997). The Art of Computer Programming, Volume 1. Addison-Wesley.

Tabla de operadores lógicos

Operador	Significado	Descripción
AND	Conjunción	Verdadero si ambas condiciones son verdaderas
Y		
OR	Disyunción	Verdadero si al menos una condición es verdadera
O		
NOT	Negación	Invierte el valor lógico de una condición
NO		

Tabla 3. Operadores lógicos.

Tabla de operadores de asignación

Operador	Significado	Descripción
← o <- o =	Asignación	Almacena un valor en una variable

Tabla 4. Operadores de asignación.

Tabla de operadores de control de flujo (decisión)

Estructura	Tipo	Descripción
Selección simple	Condicional	Ejecuta acciones solo si la condición es verdadera
Selección doble	Condicional	Define dos rutas de ejecución (verdadero/falso)
Selección múltiple	Condicional	Permite varias alternativas según una expresión

Tabla 5. Operadores de control de flujo.

Operadores simples y compuestos

Tipo de operador	Definición	Características
Simple	Operador que realiza una única operación básica sobre uno o dos operandos.	Es elemental, no combina operaciones; se usa para cálculos o comparaciones directas.[1]
Compuesto	Expresión que combina varios operadores simples en una misma instrucción.	Integra operaciones aritméticas, relacionales o lógicas para resolver problemas más complejos.[2]

Tabla 6. Operadores simples y compuestos.

[1] Sebesta, R. W. (2016). Concepts of Programming Languages (11th ed.). Pearson.

[2] Joyanes Aguilar, L. (2020). Fundamentos de Programación: Algoritmos, estructuras de datos y objetos (5.ª ed.). McGraw-Hill.

Tabla de ejemplos de operadores simples y compuestos

Tipo	Ejemplo	Descripción
Simple	+	Suma de dos valores numéricos
Simple	>	Compara si un valor es mayor que otro
Simple	AND	Evalúa dos condiciones lógicas
Simple	<-	Asignación de un valor a una variable
Compuesto	(a + b) * c	Combina suma y multiplicación en una sola expresión
Compuesto	(x > 10) AND (y < 5)	Combina dos comparaciones con operador lógico
Compuesto	((a + b) / c) >= d	Integra operaciones aritméticas y una comparación relacional
Compuesto	(x MOD 2 = 0) OR (x > 100)	Combina residuo, comparación y operador lógico

Tabla 7. Ejemplos de operadores.

Seguridad en la ejecución

Prevención de incidentes técnicos y eléctricos: Durante el uso de la computadora, si detecta chispas, calentamiento extremo al tacto en el chasis o congelamiento del sistema, notifique de inmediato al instructor y no intente apagar el equipo a la fuerza para evitar descargas o daños irreversibles.

Equipo o herramientas digitales a utilizar

- Navegador web disponible, cuando la actividad lo requiera.
- Procesador de texto.
- Herramienta para elaboración de esquemas o representaciones gráficas.
- Software para diseño y representación de algoritmos.
- Plataforma institucional para entrega de evidencias, cuando corresponda.

Guía de apoyo:

Para construir un algoritmo basado en secuencias, considera los siguientes aspectos:

Paso 1. Analizar el problema

- Identificar entradas (datos)
- Identificar salidas (resultados)

Paso 2. Definir el objetivo

- Establece claramente qué problema se desea resolver.
- ¿Qué se desea calcular o resolver?

Paso 3. Identificar acciones y pasos necesarios

- Enumera las acciones requeridas para alcanzar el objetivo.
- Listar las operaciones necesarias.

Paso 4. Ordenar las acciones de forma lógica

- Asegura que los pasos estén organizados correctamente.
- Establecer una secuencia lógica (inicio → proceso → salida)

Paso 5. Construir el algoritmo

- Redactar los pasos de forma clara y ordenada
- Utiliza una forma estructurada para describir el algoritmo.

Paso 6. Validar

- Realizar prueba de escritorio.
- Ejecuta paso a paso el algoritmo para verificar su funcionamiento.

Ejemplo guiado:

Problema: Determinar el costo de la energía eléctrica consumida por un equipo durante un periodo determinado, considerando la potencia del equipo, el tiempo de funcionamiento y el costo de la energía eléctrica.

Paso 1. Analizar el problema

- Entradas:
  - Potencia del equipo en watts.
  - Tiempo de funcionamiento en horas.
  - Costo de la energía eléctrica por kWh.

- Salida:
  - Energía consumida en kWh.
  - Costo total del consumo eléctrico.

Paso 2. Definir el objetivo

Calcular la energía eléctrica consumida y su costo total a partir de la potencia del equipo, el tiempo de funcionamiento y el costo por kilowatt-hora.

Paso 3. Identificar acciones y pasos necesarios

1. Solicitar la potencia del equipo.
2. Solicitar el tiempo de funcionamiento.
3. Solicitar el costo por kWh.
4. Convertir la potencia de watts a kilowatts.
5. Calcular la energía consumida.
6. Calcular el costo total.
7. Mostrar los resultados.

Paso 4. Ordenar las acciones de forma lógica

La secuencia será:

Inicio → ingresar datos → convertir potencia → calcular consumo → calcular costo → mostrar resultados → Fin

Las fórmulas son:

$$Potencia_{kW} = \frac{Potencia_W}{1000}$$

$$Energía_{kWh} = Potencia_{kW} \times Tiempo$$

$$Costo = Energía_{kWh} \times Costo_{kWh}$$

Paso 5. Construir el algoritmo

```
Algoritmo Consumo_Electrico

    Definir potenciaW, tiempo, costoKWh Como Real
    Definir potenciaKW, energia, costoTotal Como Real

    Escribir "Ingrese la potencia del equipo en watts:"
    Leer potenciaW

    Escribir "Ingrese el tiempo de funcionamiento en horas:"
    Leer tiempo

    Escribir "Ingrese el costo por kWh:"
    Leer costoKWh

    potenciaKW ← potenciaW / 1000
    energia ← potenciaKW * tiempo
    costoTotal ← energia * costoKWh

    Escribir "Energía consumida: ", energia, " kWh"
    Escribir "Costo total: $", costoTotal

FinAlgoritmo
```

Tabla 1. Pseudocódigo generado en PSeInt en perfil flexible.

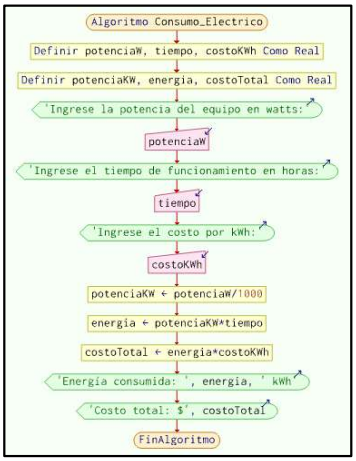


Figura 2. Diagrama de flujo construido en PSeInt en perfil flexible.

Explicación paso a paso

1. Iniciar el algoritmo.
2. Solicitar la potencia del equipo en watts.
3. Registrar la potencia proporcionada.
4. Solicitar el tiempo de funcionamiento del equipo en horas.
5. Registrar el tiempo de funcionamiento.

6. Solicitar el costo de la energía eléctrica por kilowatt-hora (kWh).
7. Registrar el costo por kWh.
8. Convertir la potencia de watts a kilowatts, dividiendo la potencia entre 1000.
9. Calcular la energía consumida, multiplicando la potencia en kilowatts por el tiempo de funcionamiento.
10. Calcular el costo total, multiplicando la energía consumida por el costo de cada kWh.
11. Mostrar la energía eléctrica consumida en kWh.
12. Mostrar el costo total del consumo eléctrico.
13. Finalizar el algoritmo.

Cada paso depende del anterior, por eso es una estructura secuencial.

Paso 6. Validar

Se ejecuta el algoritmo manualmente para verificar su funcionamiento.

Paso	Potencia W	Potencia kW	Tiempo h	Energía kWh
Entrada	1500	—	4	—
Conversión	1500	1.5	4	—
Cálculo	1500	1.5	4	6
Resultado	1500	1.5	4	6

Tabla 8. Validar ejemplo.

Resultado

Resultado: El equipo consume 6 kWh y el costo total es de $15.00.

Interpretación del resultado

Con los datos:

- Potencia: 1500 W
- Tiempo de funcionamiento: 4 h
- Costo de la energía: $2.50/kWh

Primero, la potencia se convierte de watts a kilowatts:

$$1500\ W \div 1000 = 1.5\ kW$$

Después, se calcula la energía consumida:

$$1.5\ \text{kW} \times 4\ \text{h} = 6\ \text{kWh}$$

Finalmente, se calcula el costo:

$$6\ \text{kWh} \times \$2.50 = \$15.00$$

Interpretación: El equipo, al funcionar durante 4 horas con una potencia de 1500 W, consume 6 kWh de energía, lo que representa un costo total de $15.00 al precio indicado.

Actividades propuestas:

Diseñar, representar y validar algoritmos mediante pseudocódigo y pruebas manuales.

1. **Seleccionar los problemas:** Seleccionar dos problemas simples, preferentemente relacionados con contextos de ingeniería o matemáticos.
2. **Describir los problemas:** Describir cada problema especificando el objetivo, los datos de entrada y los resultados esperados.
3. **Diseñar los algoritmos:** Diseñar la solución de cada problema mediante pseudocódigo claro, ordenado y sin ambigüedades.
4. **Revisar los algoritmos:** Verificar que los pasos sigan un orden lógico, que no existan omisiones ni redundancias y que cada instrucción sea comprensible y ejecutable.
5. **Realizar la prueba de escritorio:** Ejecutar manualmente cada algoritmo utilizando valores de prueba y registrar el comportamiento de las variables durante cada paso.
6. **Analizar los resultados:** Comparar los resultados obtenidos con los resultados esperados para verificar el funcionamiento del algoritmo.
7. **Ajustar los algoritmos:** Corregir los errores identificados y mejorar la claridad, el orden y la precisión de las soluciones.

Producto / Entregable:

Documento técnico que incluya, para cada problema planteado: la especificación del objetivo, las entradas y las salidas esperadas, el algoritmo correspondiente, el programa funcional, la tabla de prueba de escritorio con datos de ejemplo —mostrando la traza y comportamiento de las variables durante la ejecución— y los resultados obtenidos.

El entregable deberá incluir, además, una breve explicación del proceso lógico seguido para la construcción y estructuración de la solución computacional.

Referencias

Bibliografía básica

- Cairó, O. (2005). Metodología de la programación (3.ª ed.). México: Alfaomega.
- Corona, M. (2011). Diseño de algoritmos y su codificación en lenguaje C. México: McGraw-Hill.

Bibliografía complementaria

- Wing, J. M. (2006). Computational Thinking. Communications of the ACM, 49(3), 33–35.
- Zapotecatl, L. J. L. (2018). Introducción al pensamiento computacional: conceptos básicos para todos. Academia Mexicana de Computación.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.